Predicting House Price with Trulia.com Data

Sam Fortescue, Mark Gardner, Nat Koonmee

12/16/21

Abstract

The price of any commodity is often only an approximation of its true underlying value. As a result, humans, when making purchasing decisions, should take caution to avoid paying too much. In the case of real estate commodities (i.e. houses), datasets from house listing websites are plentiful; for our analysis we used a dataset from Trulia.com (PromptCloud & DataStock, 2019), which contains around 30000 house listing in the United States from September-October 2019. Statistical learning offers a well defined framework for assisting in decisions, by comparing houses considered for purchase with others that are similar. Our goal was to predict house listing price using Multiple Linear Regression, LASSO and Ridge Regression, Principal Component and Partial Least Squares Regression, Generalized Additive Models (Polynomial Regression, Step Functions), and Tree Based Methods. We subset the dataset by city and state based on the assumption that housing markets tend to be local; predicting price for homes in California, Colorado, Los Angeles, and Denver. Models were trained on 70% of the data, and predictive performance was evaluated with Mean Squared Error calculated from the remaining 30%. There was a primary problem with the data that caused issues in analysis, namely there weren't enough observations of houses that were similar to eachother. In hindsight this issue is quite obvious since house listing websites only list houses for sale, and most houses are not for sale. A contrasting dataset would be the Ames dataset (De Cock, 2011), which contains details about all houses, whether for sale or not. Ultimately the test MSE values obtained were too large for our models to be considered useful in making house purchasing decisions.

Data Wrangling

Below is a brief summary of the datasets, post-cleaning, for help in understanding. Note the variables categorized by type:

Table 1: Data summary

Name	re
Number of rows	9999
Number of columns	19
Column type frequency:	

Table 1: Data summary

character	5
factor	2
numeric	12
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Description	0	1	1	3000	0	9966	0
Address	0	1	23	57	0	9999	0
City	0	1	4	20	0	431	0
State	0	1	2	2	0	27	0
Zipcode	0	1	5	5	0	1717	0

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
Beds	0	1	FALSE	15	3: 3992, 4: 2588, 2: 1603, 5: 1015
Bath	0	1	FALSE	30	2: 3634, 3: 2718, 1: 1556, 4: 1039

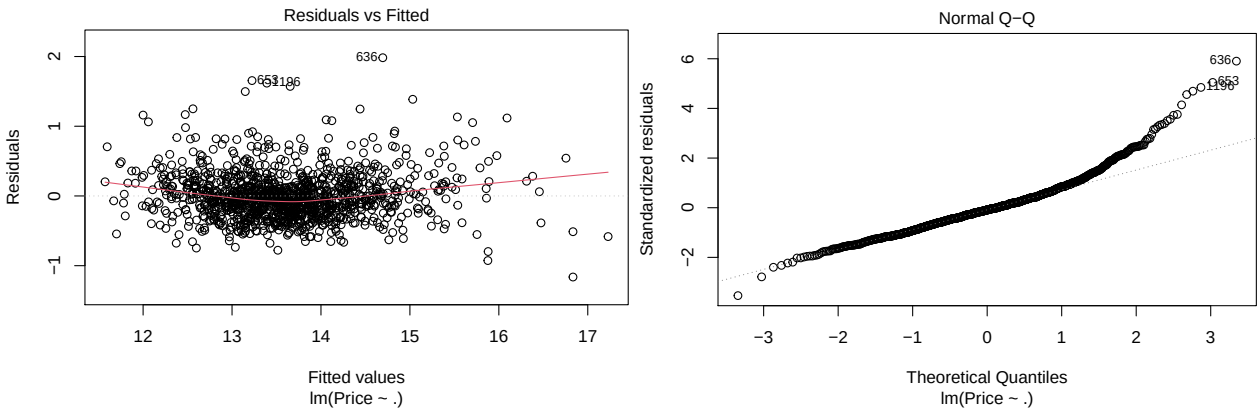
Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Price	0	1	575445.01	137220.33	3000.00	209925.00	535000.00	580000.00	32500000.0	□□□□□
SqrFt	0	1	5766.36	360037.16	1.00	1284.00	1778.00	2600.00	36003799.0	□□□□□
Longitude	0	1	-95.22	16.50	-122.84	-112.14	-87.69	-81.33	-71.0	□□□□□
Latitude	0	1	36.97	4.92	25.55	33.76	36.26	40.58	47.8	□□□□□
LotSize	0	1	0.63	15.92	0.00	0.09	0.16	0.25	1260.0	□□□□□

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
YearBuilt	0	1	1968.51	33.96	1808.00	1946.00	1972.00	1999.00	2019.0	▁▁▅▅▅
PriceSqrFt	0	1	400.70	11495.62	0.00	126.00	175.00	275.00	1100000.0	▁▁▁▁▁
LastSoldYear	0	1	2011.06	6.67	1987.00	2006.00	2013.00	2017.00	2019.0	▁▁▅▅▅
LastSoldFor	0	1	361647.95	5350636.90	1.00	117000.00	215000.00	389850.00	1200000000.0	▁▁▁▁▁
LastTaxAssestment	0	1	256225.07	800965.82	0.00	28249.00	101370.00	258823.50	34252200.0	▁▁▁▁▁
LastTaxYear	0	1	2017.84	0.85	2010.00	2018.00	2018.00	2018.00	2019.0	▁▁▅▅▅
DaysOnTrulia	0	1	56.18	58.61	2.00	6.00	35.00	90.50	180.0	▁▁▅▅▅

The Trulia.com dataset (PromptCloud & DataStock, 2019) initially had 67 columns to consider as predictors, however many didn't have any predictive meaning i.e. `Unique House ID`. In addition, several predictors contained the same information as another, and several more were entirely composed of NA values. After the removal of these columns there were 18 potentially useful predictors. Two of these were `Address` and `Description`; we considered pulling additional features out of these e.g. an indicator if a description contained the word "beautiful," or street name. Given that there usually weren't multiple house listings on the same street, and beautiful is just word that anyone can use, we opted not to do these things. After pulling out useful predictors, we removed observations that had missing observations. In hindsight this may have been a mistake because as we will see in the methodology section below, we didn't end up using many predictors due to errors in analysis, so we could have potentially had more data. The remainder of the data cleaning consisted of removing unit labels, converting all `LotSize` observations to Acres as some were in SqrFt, getting rid of dollar signs, and making everything that should be a factor a factor (or character), and everything else numeric.

For linear models it was necessary to do several data transformations. Most notably we predicted log(`Price`) instead of `Price`, however we were still able to get interpretable predictions via inverse transformations. In general for predictor variables that represented a dollar amount, a log transform worked well. For `SqrFt` taking the square root worked well. We did not transform time data e.g. `LastTaxYear`, because doing log(`Price`) was enough, also it doesn't really make sense to do so. It should be noted that we did look at potentially doing different transformations for different subsets, it just turned out that the same transformations worked for each. The following diagnostic plots for a multiple linear regression on all numeric predictors from the California subset, show that the transformations worked reasonably well at making a linear model have lower bias:



It's clear that a linear model will not capture some of the true relationship between the transformed predictors and log(`Price`), which is why we decided to fit GAMs on the transformed dataset(s) as well.

To see our exact data wrangling process view the code appendix below under "# DATA WRANGLING".

Statistical Methodology

For each set of statistical models our goal was to approximate the true relationship between the predictors in our dataset (PromptCloud, & DataStock, 2019) and list Price of houses. More formally we look for a function $\hat{f}(\mathbf{x})$ such that $\hat{f}(\mathbf{x})$ is a decent approximation of the truth as measured by predictive accuracy.

Linear Models

The line-based modeling methods considered include Multiple Linear Regression, LASSO and Ridge Regression, Principal Component and Partial Least Squares Regression. We recall that MLR minimizes residual sum of squares to find a best fitting line, and LASSO and Ridge perform a similar optimization however in each there is a penalty term which causes unnecessary β_j values to shrink. The LASSO can force β_j values to be exactly 0, whereas Ridge can only make them very small. Principal Component and Partial Least Squares regression are both dimension reduction techniques. In both cases multiple linear regression is performed not on the original predictors, but on linear combinations of the predictors.

An interesting error arose when performing multiple linear regression in which dummy variables for different categorical predictors caused linear dependencies in the design matrix. The only solution we found was to remove conflicting categorical predictors, which tended to be things that described the same information e.g. `City` and `Zipcode` were sometimes perfectly collinear. This give some insight into the potential origin of this issue as there were likely some Cities and Zipcodes in the data that shared most of

the same observations. Intuitively, having more observations could have solved this problem as well. Our decision to only predict on one of the factors seems reasonable in this instance. LASSO and Ridge regression seemed to not be affected by this issue, I'm not sure if this was the result of the R functions that we used automatically dropping dependencies like this, or if the optimization methods themselves were successful at making the β_j values associated with these dummy variables so small that the issue didn't arise.

As far as I'm aware it doesn't make any sense to use categorical predictors with dimension reduction techniques. Taking a linear combination of dummy variables would work mathematically, but would likely just produce a bad set of principal components as regressors.

Generalized Additive Models

Nat

Tree Based Models

Mark

Results

Linear Models

The test-MSEs in log-price for each subset and method are given. In addition we provide the test-RMSE after performing the inverse log transformation i.e. e^x, on observed and predicted log-price.

California

The best performing model fit for the California subset was done with the LASSO:

```
## MSE (log(Price)):  0.08228367
```

```
## RMSE (after inverse transformation):  1359257
```

Colorado

The best performing model fit for the Colorado subset was done with Ridge Regression:

```
## MSE (log(Price)):  0.03516985
```

```
## RMSE (after inverse transformation):  124241.8
```

Los Angeles

The best performing model fit for the Los Angeles subset was done with the LASSO:

```
## MSE (log(Price)):  0.1532133
```

```
## RMSE (after inverse transformation):  1883764
```

Denver

The best performing model fit for the Denver subset was done with Ridge Regression:

```
## MSE (log(Price)):  0.04000187
```

```
## RMSE (after inverse transformation):  151154
```

Generalized Additive Models

Nat

Tree Based Models

Mark

Conclusions

References

De Cock, Dean. 2011. 'Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project.' Journal of Statistics Education, Volume 19, Number 3. Retrieved 12/14/2021 from (http://jse.amstat.org/v19n3/decock.pdf)

PromptCloud, & DataStock, 2019. Real Estate Data From Trulia (Version 1). A house listing dataset from Trulia.com. Retrieved 12/14/2021 from (https://www.kaggle.com/promptcloud/real-estate-data-from-trulia)

Code Appendix

```r
# PACKAGES NEEDED

library(readr)
library(ggplot2)
library(leaps)
library(glmnet)
library(car)
library(pls)
library(dplyr)
library(knitr)
library(skimr)


# DATA WRANGLING CODE

# read in data
re <- read_csv("./realEstate.csv")

# get rid of useless columns
re <- re[,c(-(1:4),-7,-8,-12,-18,-(27:29),-(31:68))]

# remove observations with missing data
re <- na.omit(re)

# make better var names
names(re) <- gsub(" ","", names(re))

# remove unit labels and dollar signs
re$Price <- parse_number(re$Price)
re$SqrFt <- parse_number(re$SqrFt)

inSqft <- rep(NA,length(re$LotSize))
for (i in 1:length(re$LotSize)) {
  if (grepl("sqft",re$LotSize[i],fixed=TRUE)) {
    inSqft[i] <- TRUE
  } else {
    inSqft[i] <- FALSE
  }
}
re$LotSize <- parse_number(re$LotSize)
for (i in 1:length(re$LotSize)) {
  if (inSqft[i]) {
    re$LotSize[i] <- re$LotSize[i]*(1/43560)
  }
}

re$Bath <- as.factor(re$Bath)
re$Beds <- as.factor(re$Beds)

re$LastSoldFor <- parse_number(re$LastSoldFor)
re$LastTaxAssestment <- parse_number(re$LastTaxAssestment)
re$PriceSqrFt <- parse_number(re$PriceSqrFt)
```

```r
# SUBSETS

ca <- re[re$State == "CA",c(-1,-10,-15)]
co <- re[re$State == "CO",c(-1,-10,-15)]
la <- re[re$City == "Los Angeles",c(-1,-10,-15)]
den <- re[re$City == "Denver",c(-1,-10,-15)]


# DATA TRANSFORMATIONS

ca_t <- ca[,c(-7,-14,-15)]
ca_t$Price <- log(ca_t$Price)
ca_t$SqrFt <- sqrt(ca_t$SqrFt)
ca_t$LotSize <- log(ca_t$LotSize)
ca_t$LastSoldFor <- log(ca_t$LastSoldFor)
ca_t$LastTaxAssestment <- log(ca_t$LastTaxAssestment)
ca_t$LastTaxYear <- as.factor(ca_t$LastTaxYear)

co_t <- co[,c(-12,-14,-15)]
co_t$Price <- log(co_t$Price)
co_t$SqrFt <- sqrt(co_t$SqrFt)
co_t$LotSize <- log(co_t$LotSize)
co_t$LastSoldFor <- log(co_t$LastSoldFor)
co_t$LastTaxAssestment <- log(co_t$LastTaxAssestment)

la_t <- la[,c(-7,-13,-14,-15)]
la_t$Price <- log(la_t$Price)
la_t$SqrFt <- sqrt(la_t$SqrFt)
la_t$LotSize <- log(la_t$LotSize)
la_t$LastSoldFor <- log(la_t$LastSoldFor)
la_t$LastTaxAssestment <- log(la_t$LastTaxAssestment)
la_t$LastTaxYear <- as.factor(la_t$LastTaxYear)

den_t <- den[,c(-12,-13,-14,-15)]
den_t$Price <- log(den_t$Price)
den_t$SqrFt <- sqrt(den_t$SqrFt)
den_t$LotSize <- log(den_t$LotSize)
den_t$LastSoldFor <- log(den_t$LastSoldFor)
den_t$LastTaxAssestment <- log(den_t$LastTaxAssestment)


# DATA SUMMARY

skim(re)


# MLR California on Transformed Numeric Predictors

diag_lm_ca <- lm(Price~.,data=ca_t[,c(-6,-12)])


# DIAGNOSTIC PLOTS CALIFORNIA
```

```r
plot(diag_lm_ca,c(1))
plot(diag_lm_ca,c(2))


k <- 10
lambda <- 10^seq(-4,10,length.out=1000)

set.seed(420)

# California

n <- nrow(ca_t)
trn <- seq_len(n) %in% sample(seq_len(n),round(0.7*n))

n <- length(trn[trn==TRUE])
folds <- sample(seq_len(k), n, replace = k != n)

## Base Model

base_lm_ca <- lm(Price~.,data=ca_t[trn,c(-6,-12)])
aliasMatrixCA <- alias(base_lm_ca)$Complete
vif(base_lm_ca)

pred_base_ca <- predict(base_lm_ca,ca_t[!trn,c(-6,-12)])
mse_base_ca <- mean((ca_t[!trn,c(-6,-12)]$Price - pred_base_ca)^2)
inv_base_ca <- mean((exp(ca_t[!trn,c(-6,-12)]$Price) - exp(pred_base_ca))^2)

## Ridge

cv_ridge_ca <- cv.glmnet(model.matrix(Price~.,data=ca_t)[trn,-1],unlist(ca_t[trn,1]),alpha=0,lambda=lambda,fo

ridge_trn_ca <- glmnet(model.matrix(Price~.,data=ca_t)[trn,-1],unlist(ca_t[trn,1]),alpha=0,lambda=cv_ridge_ca
pred_ridge_ca <- predict(ridge_trn_ca, model.matrix(Price~.,data=ca_t)[!trn,-1])

mse_ridge_ca <- mean((ca_t[!trn,]$Price - pred_ridge_ca)^2)
inv_ridge_ca <- mean((exp(ca_t[!trn,]$Price) - exp(pred_ridge_ca))^2)

## Lasso

cv_lasso_ca <- cv.glmnet(model.matrix(Price~.,data=ca_t)[trn,-1],unlist(ca_t[trn,1]),alpha=1,lambda=lambda,fo

lasso_trn_ca <- glmnet(model.matrix(Price~.,data=ca_t)[trn,-1],unlist(ca_t[trn,1]),alpha=1,lambda=cv_lasso_ca
pred_lasso_ca <- predict(lasso_trn_ca, model.matrix(Price~.,data=ca_t)[!trn,-1])

mse_lasso_ca <- mean((ca_t[!trn,]$Price - pred_lasso_ca)^2)
inv_lasso_ca <- mean((exp(ca_t[!trn,]$Price) - exp(pred_lasso_ca))^2)

## PCR

cv_pcr_ca <- pcr(Price ~ ., data = ca_t[trn,c(-6,-7,-12,-13)], validation = "CV", scale=TRUE, segments=cvsegm
mse_cv <- MSEP(cv_pcr_ca)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]
```

```r
pcr_trn_ca <- pcr(Price ~., data = ca_t[trn,c(-6,-7,-12,-13)], scale=TRUE, M=M)
pred_pcr_ca <- predict(pcr_trn_ca, model.matrix(Price ~ .,data=ca_t[,c(-6,-7,-12,-13)])[!trn,-1])

mse_pcr_ca <- mean((ca_t[!trn,]$Price - pred_pcr_ca)^2)
inv_pcr_ca <- mean((exp(ca_t[!trn,]$Price) - exp(pred_pcr_ca))^2)

## PLS

cv_pls_ca <- plsr(Price ~ ., data = ca_t[trn, c(-6,-7,-12,-13)], validation = "CV", scale=TRUE,segments=cvseg
mse_cv <- MSEP(cv_pls_ca)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pls_trn_ca <- plsr(Price ~., data = ca_t[trn,c(-6,-7,-12,-13)], scale=TRUE, M=M)
pred_pls_ca <- predict(pls_trn_ca, model.matrix(Price ~ .,data=ca_t[,c(-6,-7,-12,-13)])[!trn,-1])

mse_pls_ca <- mean((ca_t[!trn,]$Price - pred_pls_ca)^2)
inv_pls_ca <- mean((exp(ca_t[!trn,]$Price) - exp(pred_pls_ca))^2)

errors_ca <- c(mse_base_ca,mse_ridge_ca,mse_lasso_ca,mse_pcr_ca,mse_pls_ca)
inv_errors_ca <- c(inv_base_ca,inv_ridge_ca,inv_lasso_ca,inv_pcr_ca,inv_pls_ca)


set.seed(420)

# Colorado

n <- nrow(co_t)
trn <- seq_len(n) %in% sample(seq_len(n),round(0.7*n))

n <- length(trn[trn==TRUE])
folds <- sample(seq_len(k), n, replace = k != n)

## Base Model

base_lm_co <- lm(Price~.,data=co_t[trn,c(-6,-7,-12)])
aliasMatrixCO <- alias(base_lm_co)$Complete
vif(base_lm_co)

pred_base_co <- predict(base_lm_co,co_t[!trn,c(-6,-7,-12)])
mse_base_co <- mean((co_t[!trn,c(-6,-7,-12)]$Price - pred_base_co)^2)
inv_base_co <- mean((exp(co_t[!trn,c(-6,-12)]$Price) - exp(pred_base_co))^2)

## Ridge

cv_ridge_co <- cv.glmnet(model.matrix(Price~.,data=co_t)[trn,-1],unlist(co_t[trn,1]),alpha=0,lambda=lambda,fo

ridge_trn_co <- glmnet(model.matrix(Price~.,data=co_t)[trn,-1],unlist(co_t[trn,1]),alpha=0,lambda=cv_ridge_co
pred_ridge_co <- predict(ridge_trn_co, model.matrix(Price~.,data=co_t)[!trn,-1])

mse_ridge_co <- mean((co_t[!trn,]$Price - pred_ridge_co)^2)
inv_ridge_co <- mean((exp(co_t[!trn,]$Price) - exp(pred_ridge_co))^2)

## Lasso
```

```r
cv_lasso_co <- cv.glmnet(model.matrix(Price~.,data=co_t)[trn,-1],unlist(co_t[trn,1]),alpha=1,lambda=lambda,fo

lasso_trn_co <- glmnet(model.matrix(Price~.,data=co_t)[trn,-1],unlist(co_t[trn,1]),alpha=1,lambda=cv_lasso_co
pred_lasso_co <- predict(lasso_trn_co, model.matrix(Price~.,data=co_t)[!trn,-1])

mse_lasso_co <- mean((co_t[!trn,]$Price - pred_lasso_co)^2)
inv_lasso_co <- mean((exp(co_t[!trn,]$Price) - exp(pred_lasso_co))^2)

## PCR

cv_pcr_co <- pcr(Price ~ ., data = co_t[trn,c(-6,-7,-12)], validation = "CV", scale=TRUE, segments=cvsegments
mse_cv <- MSEP(cv_pcr_co)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pcr_trn_co <- pcr(Price ~., data = co_t[trn,c(-6,-7,-12,-13)], scale=TRUE, M=M)
pred_pcr_co <- predict(pcr_trn_co, model.matrix(Price ~ .,data=co_t[,c(-6,-7,-12,-13)])[!trn,-1])

mse_pcr_co <- mean((co_t[!trn,]$Price - pred_pcr_co)^2)
inv_pcr_co <- mean((exp(co_t[!trn,]$Price) - exp(pred_pcr_co))^2)

## PLS

cv_pls_co <- plsr(Price ~ ., data = co_t[trn, c(-6,-7,-12)], validation = "CV", scale=TRUE,segments=cvsegment
mse_cv <- MSEP(cv_pls_co)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pls_trn_co <- plsr(Price ~., data = co_t[trn,c(-6,-7,-12)], scale=TRUE, M=M)
pred_pls_co <- predict(pls_trn_co, model.matrix(Price ~ .,data=co_t[,c(-6,-7,-12)])[!trn,-1])

mse_pls_co <- mean((co_t[!trn,]$Price - pred_pls_co)^2)
inv_pls_co <- mean((exp(co_t[!trn,]$Price) - exp(pred_pls_co))^2)

errors_co <- c(mse_base_co,mse_ridge_co,mse_lasso_co,mse_pcr_co,mse_pls_co)
inv_errors_co <- c(inv_base_co,inv_ridge_co,inv_lasso_co,inv_pcr_co,inv_pls_co)


set.seed(420)

# Los Angeles

n <- nrow(la_t)
trn <- seq_len(n) %in% sample(seq_len(n),round(0.7*n))

n <- length(trn[trn==TRUE])
folds <- sample(seq_len(k), n, replace = k != n)

## Base Model

base_lm_la <- lm(Price~.,data=la_t[trn,c(-6,-11)])
aliasMatrixLA <- alias(base_lm_la)$Complete
vif(base_lm_la)

pred_base_la <- predict(base_lm_la,la_t[!trn,c(-6,-11)])
```

```r
mse_base_la <- mean((la_t[!trn,c(-6,-12)]$Price - pred_base_la)^2)
inv_base_la <- mean((exp(la_t[!trn,c(-6,-12)]$Price) - exp(pred_base_la))^2)

## Ridge

cv_ridge_la <- cv.glmnet(model.matrix(Price~.,data=la_t)[trn,-1],unlist(la_t[trn,1]),alpha=0,lambda=lambda,fo

ridge_trn_la <- glmnet(model.matrix(Price~.,data=la_t)[trn,-1],unlist(la_t[trn,1]),alpha=0,lambda=cv_ridge_la
pred_ridge_la <- predict(ridge_trn_la, model.matrix(Price~.,data=la_t)[!trn,-1])

mse_ridge_la <- mean((la_t[!trn,]$Price - pred_ridge_la)^2)
inv_ridge_la <- mean((exp(la_t[!trn,]$Price) - exp(pred_ridge_la))^2)

## Lasso

cv_lasso_la <- cv.glmnet(model.matrix(Price~.,data=la_t)[trn,-1],unlist(la_t[trn,1]),alpha=1,lambda=lambda,fo

lasso_trn_la <- glmnet(model.matrix(Price~.,data=la_t)[trn,-1],unlist(la_t[trn,1]),alpha=1,lambda=cv_lasso_la
pred_lasso_la <- predict(lasso_trn_la, model.matrix(Price~.,data=la_t)[!trn,-1])

mse_lasso_la <- mean((la_t[!trn,]$Price - pred_lasso_la)^2)
inv_lasso_la <- mean((exp(la_t[!trn,]$Price) - exp(pred_lasso_la))^2)

## PCR

cv_pcr_la <- pcr(Price ~ ., data = la_t[trn,c(-6,-11)], validation = "CV", scale=TRUE, segments=cvsegments(nr
mse_cv <- MSEP(cv_pcr_la)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pcr_trn_la <- pcr(Price ~., data = la_t[trn,c(-6,-11)], scale=TRUE, M=M)
pred_pcr_la <- predict(pcr_trn_la, model.matrix(Price ~ .,data=la_t[,c(-6,-11)])[!trn,-1])

mse_pcr_la <- mean((la_t[!trn,]$Price - pred_pcr_la)^2)
inv_pcr_la <- mean((exp(la_t[!trn,]$Price) - exp(pred_pcr_la))^2)

## PLS

cv_pls_la <- plsr(Price ~ ., data = la_t[trn, c(-6,-11)], validation = "CV", scale=TRUE,segments=cvsegments(n
mse_cv <- MSEP(cv_pls_la)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pls_trn_la <- plsr(Price ~., data = la_t[trn,c(-6,-11)], scale=TRUE, M=M)
pred_pls_la <- predict(pls_trn_la, model.matrix(Price ~ .,data=la_t[,c(-6,-11)])[!trn,-1])

mse_pls_la <- mean((la_t[!trn,]$Price - pred_pls_la)^2)
inv_pls_la <- mean((exp(la_t[!trn,]$Price) - exp(pred_pls_la))^2)

errors_la <- c(mse_base_la,mse_ridge_la,mse_lasso_la,mse_pcr_la,mse_pls_la)
inv_errors_la <- c(inv_base_la,inv_ridge_la,inv_lasso_la,inv_pcr_la,inv_pls_la)


set.seed(420)
```

```r
# Denver

n <- nrow(den_t)
trn <- seq_len(n) %in% sample(seq_len(n),round(0.7*n))

n <- length(trn[trn==TRUE])
folds <- sample(seq_len(k), n, replace = k != n)

## Base Model

base_lm_den <- lm(Price~.,data=den_t[trn,c(-6,-7)])
aliasMatrixDEN <- alias(base_lm_den)$Complete
vif(base_lm_den)

pred_base_den <- predict(base_lm_den,den_t[!trn,c(-6,-7)])
mse_base_den <- mean((den_t[!trn,c(-6,-7)]$Price - pred_base_den)^2)
inv_base_den <- mean((exp(den_t[!trn,c(-6,-12)]$Price) - exp(pred_base_den))^2)

## Ridge

cv_ridge_den <- cv.glmnet(model.matrix(Price~.,data=den_t)[trn,-1],unlist(den_t[trn,1]),alpha=0,lambda=lambda

ridge_trn_den <- glmnet(model.matrix(Price~.,data=den_t)[trn,-1],unlist(den_t[trn,1]),alpha=0,lambda=cv_ridge
pred_ridge_den <- predict(ridge_trn_den, model.matrix(Price~.,data=den_t)[!trn,-1])

mse_ridge_den <- mean((den_t[!trn,]$Price - pred_ridge_den)^2)
inv_ridge_den <- mean((exp(den_t[!trn,]$Price) - exp(pred_ridge_den))^2)

## Lasso

cv_lasso_den <- cv.glmnet(model.matrix(Price~.,data=den_t)[trn,-1],unlist(den_t[trn,1]),alpha=1,lambda=lambda

lasso_trn_den <- glmnet(model.matrix(Price~.,data=den_t)[trn,-1],unlist(den_t[trn,1]),alpha=1,lambda=cv_lasso
pred_lasso_den <- predict(lasso_trn_den, model.matrix(Price~.,data=den_t)[!trn,-1])

mse_lasso_den <- mean((den_t[!trn,]$Price - pred_lasso_den)^2)
inv_lasso_den <- mean((exp(den_t[!trn,]$Price) - exp(pred_lasso_den))^2)

## PCR

cv_pcr_den <- pcr(Price ~ ., data = den_t[trn,c(-6,-7)], validation = "CV", scale=TRUE, segments=cvsegments(
mse_cv <- MSEP(cv_pcr_den)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pcr_trn_den <- pcr(Price ~., data = den_t[trn,c(-6,-7)], scale=TRUE, M=M)
pred_pcr_den <- predict(pcr_trn_den, model.matrix(Price ~ .,data=den_t[,c(-6,-7)])[!trn,-1])

mse_pcr_den <- mean((den_t[!trn,]$Price - pred_pcr_den)^2)
inv_pcr_den <- mean((exp(den_t[!trn,]$Price) - exp(pred_pcr_den))^2)

## PLS

cv_pls_den <- plsr(Price ~ ., data = den_t[trn, c(-6,-7)], validation = "CV", scale=TRUE,segments=cvsegments(
```

```r
mse_cv <- MSEP(cv_pls_den)
M <- mse_cv$comps[which.min(as.data.frame(mse_cv$val)[1,])]

pls_trn_den <- plsr(Price ~., data = den_t[trn,c(-6,-7)], scale=TRUE, M=M)
pred_pls_den <- predict(pls_trn_den, model.matrix(Price ~ .,data=den_t[,c(-6,-7)])[!trn,-1])

mse_pls_den <- mean((den_t[!trn,]$Price - pred_pls_den)^2)
inv_pls_den <- mean((exp(den_t[!trn,]$Price) - exp(pred_pls_den))^2)

errors_den <- c(mse_base_den,mse_ridge_den,mse_lasso_den,mse_pcr_den,mse_pls_den)
inv_errors_den <- c(inv_base_den,inv_ridge_den,inv_lasso_den,inv_pcr_den,inv_pls_den)


cat("MSE (log(Price)): ",min(errors_ca),"\n")
cat("RMSE (after inverse transformation): ",sqrt(min(inv_errors_ca)),"\n")


cat("MSE (log(Price)): ",min(errors_co),"\n")
cat("RMSE (after inverse transformation): ",sqrt(min(inv_errors_co)),"\n")


cat("MSE (log(Price)): ",min(errors_la),"\n")
cat("RMSE (after inverse transformation): ",sqrt(min(inv_errors_la)),"\n")


cat("MSE (log(Price)): ",min(errors_den),"\n")
cat("RMSE (after inverse transformation): ",sqrt(min(inv_errors_den)),"\n")
```